UNIVERSIDAD SERGIO ARBOLEDA

CIENCIAS DE LA COMPUTACION E INTELIGENCIA ARTIFICIAL

Entrega de Taller Parcial

Materia: Lenguajes de Programacion

Profesor: Joaquin Fernando

Presentado por:

Cristian Andres Reinales Herrera

17 de abril de 2026

Ejercicio 1 y 2

Enunciado:
Diseñe una gramatica de un lenguaje de programacion que permita hacer las operaciones de CRUD en una base de datos no relacional. Implementar la gramática BISON o ANTLR y realice pruebas sobre el lenguaje.

Explicacion del problema

El objetivo de este ejercicio es diseñar un Lenguaje para una base de datos no relacional que a diferencia de SQL, que esta pensado para bases de datos relacionales osea, tablas, filas y columnas. Una base de datos NoSQL trabaja con colecciones y documentos, estructuras muy similares a objetos JSON.

Por esto, lo que hare sera inventar las reglas sintacticas formales para que pueda Crear, Buscar, Actualizar y Borrar (CRUD) documentos dentro de colecciones.

Solucion

Para resolver este problema, utilice ANTLR4 junto con python para poder generar y probar la gramatica. La gramatica que implemente tiene la siguiente estructura:

Listing 1: Gramatica tipo JSON

```
grammar Ejercicio1;

// Reglas sintacticas
programa                : sentencia+ EOF ;

sentencia               : stCrear
                        | stBuscar
                        | stActualizar
                        | stEliminar ;

stCrear          : 'GUARDAR' objetoJson 'EN' IDENTIFICADOR ';' ;

stBuscar         : 'BUSCAR' 'EN' IDENTIFICADOR ('DONDE' condicion)? ';' ;

stActualizar     : 'ACTUALIZAR' IDENTIFICADOR 'CON' objetoJson ('DONDE' condicion)? ';' ;

stEliminar       : 'ELIMINAR' 'DE' IDENTIFICADOR ('DONDE' condicion)? ';' ;

// Estructura de datos
objetoJson                 : '{' (parClaveValor (',' parClaveValor)*)? '}' ;
parClaveValor           : CADENA ':' valor ;
valor                   : CADENA | NUMERO | BOOLEANO | objetoJson ;

```

```
condicion                : IDENTIFICADOR op=('<'|'>'|'=='|'!='|'<='|'>=') valor ;

// Reglas lexicas
BOOLEANO                 : 'verdadero' | 'falso' ;
IDENTIFICADOR            : [a-zA-Z_][a-zA-Z0-9_]* ;
CADENA                   : '"' ~'"'* '"' ; // Textos entre comillas
NUMERO                   : [0-9]+ ('.' [0-9]+)? ;
WS                       : [ \t\r\n]+ -> skip ; // Ignorar espacios en blanco
```

Dentro de nuestra gramatica definimos palabras clave como GUARDAR, BUSCAR, ACTUALIZAR y ELIMINAR. Con esto tenemos las 4 operaciones del CRUD.

Como el enfoque es NoSQL, creamos la estructura Json, con reglas especificas en la gramatica (`objetoJson` y `parClaveValor`) para que el lenguaje acepte la creacion y actualizacion de datos entre llaves { }, con esto nos aseguramos de estar trabajando sobre una base de datos no relacional.

Las sentencias son la gramatica de este programa, definen las regleas y como se escriben siguiendo unas reglas del formato que especificamos para trabajar con consultas CRUD. 'GUARDAR' y demas definen palabras exactas que tienen que estar en la estructura.

De esta manera, la gramatica define la estructura correcta del lenguaje, y nuestro Visitor en Python se encarga de recorrer el arbol sintactico generado, simulando la base de datos en memoria e imprimiendo el paso a paso de cada accion.

Listing 2: Logica del lenguaje - Visitor

```python
def visitStCrear(self, ctx: Ejercicio1Parser.StCrearContext):
        coleccion = ctx.IDENTIFICADOR().getText()
        archivo = self.visit(ctx.objetoJson())

        if coleccion not in self.db:
            self.db[coleccion] = []
        self.db[coleccion].append(archivo)
        print("")
        print("Creando archivo:")
        print("Archivo: ", archivo)
        print("Archivo creado en ", coleccion)
        print("")

    def visitStBuscar(self, ctx: Ejercicio1Parser.StBuscarContext):
        coleccion = ctx.IDENTIFICADOR().getText()
        registros = self.db.get(coleccion, [])

        if ctx.condicion():
            campo, op, valor = self.visit(ctx.condicion())
            resultados = [r for r in registros if self.evaluar(r.get(campo), op, valor)]
        else:
```

```python
            resultados = registros
        print("")
        print("Busqueda de archivo:")
        print("Resultado: ", resultados)
        print("")

    def visitStActualizar(self, ctx: Ejercicio1Parser.StActualizarContext):
        coleccion = ctx.IDENTIFICADOR().getText()
        nuevos_datos = self.visit(ctx.objetoJson())
        registros = self.db.get(coleccion, [])

        actualizados = 0
        for r in registros:
            if not ctx.condicion() or self.evaluar(r.get(self.visit(ctx.condicion())[0]), self.visit(ctx.condicion())[1], self.visit(ctx.condicion())[2]):
                r.update(nuevos_datos)
                actualizados += 1
        print("")
        print("Actualizando datos")
        print("Se actualizo: ", actualizados)
        print("De la bd: ", coleccion)
        print("")



    def visitStEliminar(self, ctx: Ejercicio1Parser.StEliminarContext):
        coleccion = ctx.IDENTIFICADOR().getText()
        registros = self.db.get(coleccion, [])
        print("")
        print("Proceso de eliminar")
        if ctx.condicion():
            campo, op, valor = self.visit(ctx.condicion())
            self.db[coleccion] = [r for r in registros if not self.evaluar(r.get(campo), op, valor)]

            print(f"Documentos borrados en '{coleccion}' donde {campo} {op} {valor}.")
        else:
            self.db[coleccion] = []
            print("Todos los documentos borrados de: ", coleccion)
        print("")



    def visitObjetoJson(self, ctx: Ejercicio1Parser.ObjetoJsonContext):
        obj = {}
```

```
        for keyval_ctx in ctx.parClaveValor():
            clave, valor = self.visit(keyval_ctx)
            obj[clave] = valor
        return obj
```

Esta no es la version completa del archivo visitor, pero es el codigo que contiene la logica para que nuestro lenguaje pueda hacer las operaciones de CRUD.

Para la logica CRUD lo que hicimos fue crear las funciones: visitStCrear(), visitStBuscar(), visitStActualizar(), visitStEliminar(). La logica dentro de estas funciones es bastante sencilla, ya que se apoya de funciones nativas de python para hacer busquedas en los diccionarios y asi.

Pruebas sobre el lenguaje

```
Codigo fuente:
    GUARDAR { "nombre": "Cristian", "edad": 23, "activo": verdadero } EN usuarios;
    GUARDAR { "nombre": "Ana", "edad": 32, "activo": falso } EN usuarios;

    BUSCAR EN usuarios DONDE edad > 30;

    ACTUALIZAR usuarios CON { "activo": verdadero } DONDE edad > 30;
    BUSCAR EN usuarios;

    ELIMINAR DE usuarios DONDE nombre == "Cristian";
    BUSCAR EN usuarios;

=> Ejecucion:
Creando archivo:
Archivo:  {'nombre': 'Cristian', 'edad': 23, 'activo': True}
Archivo creado en  usuarios

Creando archivo:
Archivo:  {'nombre': 'Ana', 'edad': 32, 'activo': False}
Archivo creado en  usuarios

Busqueda de archivo:
Resultado:  [{'nombre': 'Ana', 'edad': 32, 'activo': False}]

Actualizando datos
Se actualizo:  1
De la bd:  usuarios

Busqueda de archivo:
Resultado:  [{'nombre': 'Cristian', 'edad': 23, 'activo': True}, {'nombre': 'Ana', 'e

Proceso de eliminar
Documentos borrados en 'usuarios' donde nombre == Cristian.

Busqueda de archivo:
Resultado:  [{'nombre': 'Ana', 'edad': 32, 'activo': True}]
```

Ejercicio 3

Enunciado:
Demuestre que la siguiente gramática:

$S \longrightarrow AaAb|BbBa$
$A \longrightarrow \varepsilon$
$B \longrightarrow \varepsilon$

Es una gramática LL(1). Utilice las herramientas que considere necesaria para hacer la demostración.

Comprobamos si es una gramatica LL(1)

Para demostrar que una gramatica es LL(1), debemos calcular los conjuntos Primeros y Siguientes de cada variable, y luego comprobar que no existen ambiguedades aplicando las reglas de interseccion.

Paso 1: Calculamos de los conjuntos de los Primeros

El conjunto de los Primeros indica con cuales simbolos terminales puede iniciar.

- **Para la variable A:** Su unica produccion es la cadena vacia ε.

$$Primeros(A) = \{\varepsilon\}$$

- **Para la variable B:** Su unica produccion es ϵ.

$$PrimerosB) = \{\varepsilon\}$$

- **Para la variable S:** Tiene dos posibles derivaciones ($AaAb$ y $BbBa$).
  - En el camino $AaAb$: Como $Primeros(A)$ contiene ε, la variable A puede desaparecer, dejando al terminal 'a' como el primer caracter visible. Asi, $Primeros(AaAb) = \{a\}$.
  - En el camino $BbBa$: Como $Primeros(B)$ contiene ε, la variable B puede desaparecer, dejando al terminal 'b' como el primer caracter visible. Asi, $Primeros(BbBa) = \{b\}$.

  En conjunto:

$$Primeros(S) = \{a, b\}$$

Paso 2: Calculo de los conjuntos Siguientes

El conjunto los siguientes indica que terminales pueden aparecer inmediatamente a la derecha de una variable durante cualquier derivacion, no incluye ε.

- **Para la variable S:** Al ser el simbolo inicial, siempre incluye el simbolo que indica el fin de la cadena. ($).

$$Siguientes(S) = \{\$\}$$

- **Para la variable A:** Buscamos donde aparece A a la derecha de las reglas.

$$S \to AaAb.$$

  - Despues de la primera A, aparece el terminal 'a'.
  - Despues de la segunda A, aparece el terminal 'b'.

$$Siguientes(A) = \{a, b\}$$

- **Para la variable B:** Buscamos donde aparece B a la derecha de las reglas.

$$S \to BbBa.$$

  - Inmediatamente despues de la primera B, aparece el terminal 'b'.
  - Inmediatamente despues de la segunda B, aparece el terminal 'a'.

$$Siguientes(B) = \{a, b\}$$

Paso 3: Calculo de los Conjuntos de Prediccion y Verificacion LL(1)

El conjunto de prediccion, es un conjunto cuyos los elementos son las compuesto por los simbolos terminales que entiende para seguir las reglas de la gramatica. Estp para saber exactamente que camino tomar sin tener que adivinar.

Para calcular el conjunto de prediccion $X \to \alpha$:

- Si α no puede derivar en la cadena vacia, el conjunto de prediccion equivale al $Primeros(\alpha)$.
- Si α puede derivar en ϵ, el conjunto de prediccion estara formado por el $Primeros(\alpha)$ excluyendo ϵ, unido al conjunto $Siguientes(X)$.

En la gramatica del ejercicio, la variable S es la unica con producciones alternativas:

- **Para la produccion $S \to AaAb$:**
  Dado que la variable A es anulable, $Primeros(A)$ contiene ϵ, el primer simbolo terminal garantizado en la derivacion es 'a'.

$$Pred(S \to AaAb) = \{a\}$$

- **Para la produccion $S \to BbBa$:**
  De manera analoga, la variable B es anulable, $Primeros(B)$ contiene ϵ, por lo que el primer simbolo en la derivacion es 'b'.

$$Pred(S \to BbBa) = \{b\}$$

Conclusion de la demostracion:

La condicion necesaria y suficiente para que una gramatica sea LL(1) es que, para cualquier variable con multiples producciones, la interseccion de sus conjuntos de prediccion sea el conjunto vacio ($\emptyset$).

Comprobamos la condicion para la variable S:

$$Pred(S \to AaAb) \cap Pred(S \to BbBa) = \{a\} \cap \{b\} = \emptyset$$

Como la interseccion es vacia, no existe colision entre los simbolos de preanalisis. **Por lo tanto, queda demostrado que la gramatica es LL(1)**.

En clases pasadas implementamos un Analizador Sintactico Descendente Recursivo en clase, el cual tiene la capacidad de a partir de una gramatica, genera los conjuntos de primeros, conjunto de siguientes y conjunto de precdiccion, ademas de poder determinar si una gramatica es o no LL1.

Al pasarle como parametros la gramatica que estamos analizando podremos comprobar computacionalmente si esta gramatica es o no LL(1).

Prueba computacional con ASDR

```
=> Calculo de primeros/segundos/predicciones y Comprobacion LL1

 Conjunto de primeros
Primeros: (A) = {'vacio'}
Primeros: (B) = {'vacio'}
Primeros: (S) = {'a', 'b'}

Conjunto de siguientes
Siguientes: (A) = {'a', 'b'}
Siguientes: (B) = {'a', 'b'}
Siguientes: (S) = {'$'}

Conjuntode predicciones
Predicciones: (S -> A a A b             )
Predicciones: (S -> B b B a             )
Predicciones: (A -> vacio               )
Predicciones: (B -> vacio               )

Es una gramatica LL1?
  La gram tica Si es una gramatica LL(1)
```

Ejercicio 4

Enunciado:
Implemente un parser usando el algoritmo CYK para realizar las operaciones de una calculadora. Realice pruebas sobre el rendimiento de este algoritmo comparándolo con un parser de tipo predictivo. Realice una comparación entre el rendimiento de los dos parser.

0.1 Algoritmo CYK - Parser Ascendente

Para dar solucion a este ejercicio, implemente el algoritmo CYK en python. Este analizador sintactico nos permitira procesar y validar expresiones matematicas como calculadora basica. El objetivo principal es medir su tiempo de ejecucion su rendimiento computacional.

El algoritmo CYK es un analizador de tipo ascendente. Para que este metodo sea capaz de aceptar y evaluar una cadena de texto, exige como requisito estricto que la gramatica sea de la forma normal de Chomsky.

Para poder hacer pruebas de rendimiento, la gramatica de la calculadora esta completamente adaptada a esta Forma Normal de Chomsky. Para simplificar el codigo definimos que cualquier numero sera representado genericamente por el simbolo terminal n, mientras que los operadores aritmeticos se mantienen como + y *. La gramatica se dfinio asi:

$$
\begin{aligned}
E &\rightarrow EA_1 \mid EA_2 \mid n \\
A_1 &\rightarrow PE \\
A_2 &\rightarrow ME \\
P &\rightarrow + \\
M &\rightarrow *
\end{aligned}
$$

Listing 3: Algoritmo CYK

```python
import time
import matplotlib.pyplot as plt

# Gramatica en forma normal de Chomsky
gramatica_cyk = {
    'E': [['E', 'A1'], ['E', 'A2'], ['n']],
    'A1': [['P', 'E']],
    'A2': [['M', 'E']],
    'P': [['+']],
    'M': [['*']]
}

def parser_cyk(cadena, gramatica, simbolo_inicial='E'): # Algoritmo CYK
    n = len(cadena)
    if n == 0:
```

```python
        return False

    tabla = [[set() for _ in range(n)] for _ in range(n)]

    for i in range(n):
        terminal = cadena[i]
        for variable, producciones in gramatica.items():
            for prod in producciones:
                if len(prod) == 1 and prod[0] == terminal:
                    tabla[i][i].add(variable)

    for longitud in range(2, n + 1):
        for i in range(n - longitud + 1):
            j = i + longitud - 1
            for k in range(i, j):
                conjunto_izquierdo = tabla[i][k]
                conjunto_derecho = tabla[k + 1][j]

                for variable, producciones in gramatica.items():
                    for prod in producciones:
                        if len(prod) == 2:
                            B = prod[0]
                            C = prod[1]
                            if B in conjunto_izquierdo and C in conjunto_derecho:
                                tabla[i][j].add(variable)

    return simbolo_inicial in tabla[0][n - 1]

def generar_cadena_prueba(n_operaciones):
    cadena = "n"
    for i in range(n_operaciones):
        cadena += "+n" if i % 2 == 0 else "*n"
    return cadena

def ejecutar_pruebas_y_graficar_cyk():
    tamanos = [5, 10, 15, 20, 25, 30, 40, 50]
    tiempos_cyk = []
    longitudes_cadena = []

    for tam in tamanos:
        cadena = generar_cadena_prueba(tam)
        longitudes_cadena.append(len(cadena))

        inicio_cyk = time.perf_counter()
        parser_cyk(cadena, gramatica_cyk)
        fin_cyk = time.perf_counter()

        tiempo_ms = (fin_cyk - inicio_cyk) * 1000
        tiempos_cyk.append(tiempo_ms)

```

```
        print("Longitud cadena: ", cadena)
        print("Tiempo de ejecucion: ", tiempo_ms)
        print("")

if __name__ == '__main__':
    ejecutar_pruebas_y_graficar_cyk()
```

Nuestra metodologia para evaluar el rendimiento consistira en pasar el algoritmo por una serie de pruebas. se basa en generar de forma automatica cadenas matematicamente validas que incrementan su longitud de manera progresiva, comenzando con expresiones muy cortas como n+n*n y aumentando gradualmente hasta alcanzar secuencias muuy largas. Al pasarle cada una de estas cadenas al analizador CYK. Con esto podremos saber como se comporta el rendimiento del algoritmo a medida que el tamaño de la cadena de entrada crece.

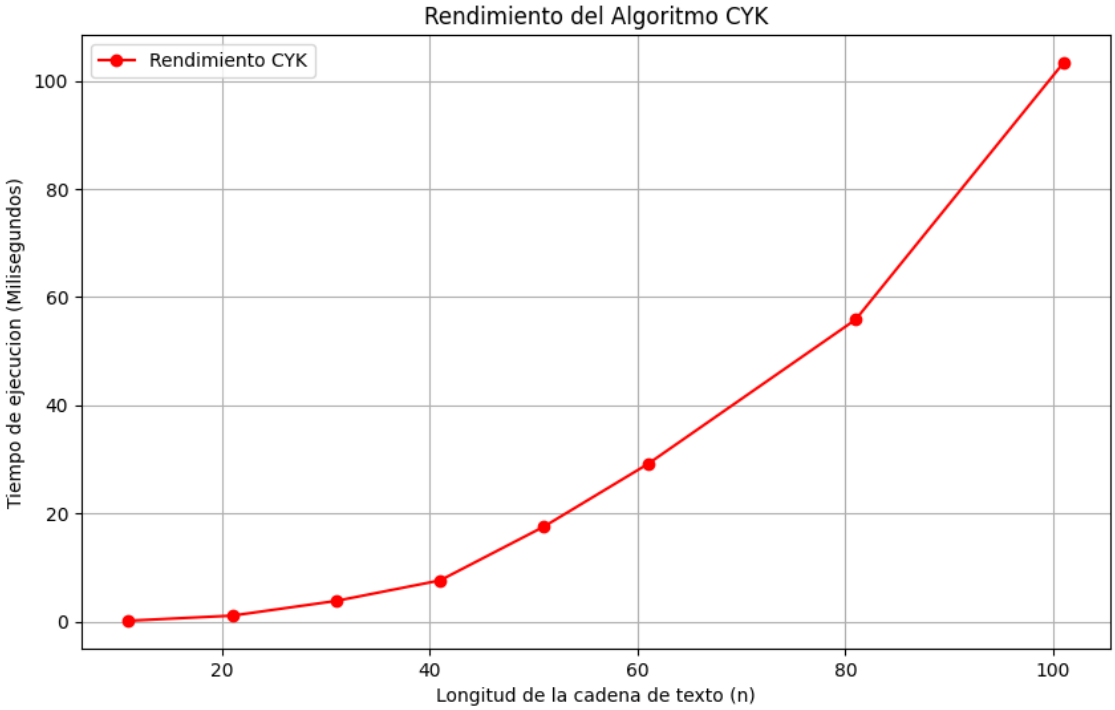

Figure 1: Grafica con tiempo de ejecucion Exponencial.

0.2 Algoritmo ASDR - Parser Predictivo

Para continuar con nuestro analisis, programamos un ASDR en Python, esto por que un analizador sintactico descendente recursivo tipo LL(1) es un parser predictivo, y con esto podremos hacer una comparacion entre los tiempos de ejecucion de lo los dos tipos de parser.

Hay algo clave que no se nos puede olvidar. Un ASDR o una gramatica de tipo LL(1), tiene 3 reglas que deben cumplir las gramaticas para poder ser procesadas.

- 1. Ausencia de recursividad por la izquierda.
- 2. Si puede ser factorizada por la izquierda, debe ser factorizada.
- 3. No ser ambigua.

Por esto mismo, si vemos la gramatica que utilizamos para CYK veremos que tiene una recursividad por la izquieda en $E\& \rightarrow EA_1 \mid EA_2 \mid n$. Vamos a eliminar la recursividad y obtener una gramatica LL(1).

Eliminacion de Recursividad por la Izquierda

Para eliminar la recursividad por la izquierda, utilizamos la formula general que dice que una produccion de la forma $A \rightarrow A\alpha \mid \beta$ se debe transformar en dos nuevas reglas: $A \rightarrow \beta A'$ y $A' \rightarrow \alpha A' \mid \epsilon$.

$$E \rightarrow EA_1 \mid EA_2 \mid n$$

Al alinear nuestra regla con la formula $A = E$ y deducimos los siguientes componentes:

- Las partes recursivas son $\alpha_1 = A_1$ y $\alpha_2 = A_2$.
- El caso base es $\beta = n$.

Sustituyendo estos valores de β y α en las formulas de transformacion. Conseguimosestas dos de derivacion:

$$\begin{aligned} E &\rightarrow nE' \\ E' &\rightarrow A_1E' \mid A_2E' \mid \epsilon \end{aligned}$$

Las demas reglas de derivacion de nuestra gramatica original no tienen recursividad por la izquierda o hay alguna factorizacion posible, por lo cual la gramatica final es:.

$$\begin{aligned} E &\rightarrow nE' \\ E' &\rightarrow A_1E' \mid A_2E' \mid \epsilon \\ A_1 &\rightarrow PE \\ A_2 &\rightarrow ME \\ P &\rightarrow + \\ M &\rightarrow * \end{aligned}$$

Con esta nueva estructura el algoritmo predictivo podra analizar cualquier cadena que sea como las cadenas de las calculadoras como $n + n \cdot n + n \cdot +n$.. Aclaro que de dejo hasta ahi la funcionabilidad de la calculadora por que la idea principal es poder comparar el rendimiento entre analizadore un analizador sintactico descendente recursivo y un analizador ascendente.

Listing 4: Algoritmo ASDR

```
import time
import matplotlib.pyplot as plt

class ParserError(Exception):
    pass

class ASDR_Calculadora:
    def __init__(self, cadena):
        self.tokens = list(cadena) + ['$']
        self.pos = 0
        self.pila = ['$', 'E']

        self.reglas = {
            'E': [['n', 'Ep']],
            'Ep': [['A1', 'Ep'], ['A2', 'Ep'], ['vacio']],
            'A1': [['P', 'E']],
            'A2': [['M', 'E']],
            'P': [['+']],
            'M': [['*']]
        }

        self.tabla_prediccion = {
            ('E', 'n'): self.reglas['E'][0],
            ('Ep', '+'): self.reglas['Ep'][0],
            ('Ep', '*'): self.reglas['Ep'][1],
            ('Ep', '$'): self.reglas['Ep'][2],
            ('A1', '+'): self.reglas['A1'][0],
            ('A2', '*'): self.reglas['A2'][0],
            ('P', '+'): self.reglas['P'][0],
            ('M', '*'): self.reglas['M'][0]
        }

    def parse(self): #Parser predictivo
        while len(self.pila) > 0:
            tope = self.pila.pop()
            actual = self.tokens[self.pos]

            if tope in ['n', '+', '*', '$']:
                if tope == actual:
                    self.pos += 1
                else:
                    raise ParserError(f"Se esperaba '{tope}' pero se encontro '{actual}'")
            elif tope == 'vacio':
```

```python
                pass
            else:
                if (tope, actual) in self.tabla_prediccion:
                    produccion = self.tabla_prediccion[(tope, actual)]
                    for simbolo in reversed(produccion):
                        self.pila.append(simbolo)
                else:
                    raise ParserError("Error de sintaxis en: " + str(actual))
        return True

def generar_cadena_prueba(n_operaciones):
    cadena = "n"
    for i in range(n_operaciones):
        cadena += "+n" if i % 2 == 0 else "*n"
    return cadena

def parser_predictivo_wrapper(cadena):
    parser = ASDR_Calculadora(cadena)
    try:
        parser.parse()
        return True
    except ParserError:
        return False

def ejecutar_pruebas_y_graficar_ll1():
    tamanos = [5, 10, 15, 20, 25, 30, 40, 50]
    tiempos_ll1 = []
    longitudes_cadena = []

    for tam in tamanos:
        cadena = generar_cadena_prueba(tam)
        longitudes_cadena.append(len(cadena))

        inicio = time.perf_counter()
        parser_predictivo_wrapper(cadena)
        fin = time.perf_counter()

        tiempo_ms = (fin - inicio) * 1000
        tiempos_ll1.append(tiempo_ms)

        print("Longitud cadena: ", cadena)
        print("Tiempo ejecucion LL1: ", tiempo_ms)
        print("")

if __name__ == '__main__':
    ejecutar_pruebas_y_graficar_ll1()
```

Al ejecutar nuestro programa y hacer pruebas con cadenas igual de largas que las que probamos en el algoritmo CYK obtenemos la siguiente grafica:

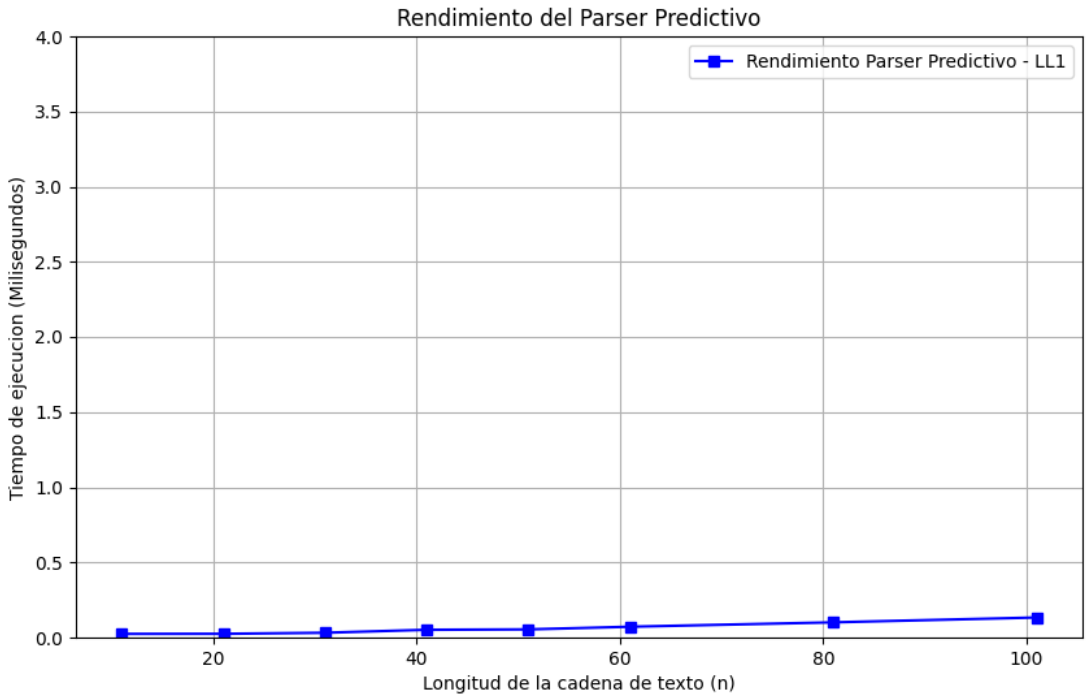

Figure 2: Grafica con tiempo de ejecucion Lineal

Los resultados reflejados en las graficas revelan una diferencia en el rendimiento computacional. El analizador ascendente CYK tiene una complejidad cubica de $O(n^3)$. Como se ve en la grafica, el tiempo de ejecucion subee rapidamente a medida que la cadena se hace mas grande.

Por otro lado, el analizador descendente predictivo LL(1) tiene una complejidad temporal lineal de $O(n)$. Es por esto que en la segunda grafica, la linea se mantiene practicamente plana y cerca a cero.

Como conclusion se ve una gran diferencia en cuanto a la eficiencia computacional.

1 Ejercicio 5

Enunciado:

Escriba un programa 'calculadora de escritorio' en YACC que evalúe expresiones booleanas. Explique como es el desempeño del analizador sintáctico para este programa.

1.1 YACC

YACC es un programa creado en los años 70 para los antiguos sistemas UNIX. Bison es la version moderna y mejorada. Bison fue diseñado para ser 100% compatible con YACC

Sabiendo esto, podemos diseñar una gramatica en Bison para implementar la calculadora de expresiones booleanas.

El primer paso es definir el archivo con el lexico de esta gramatica, esto con el fin de crear las palabras reservadas y asignar las reglas para los tokens que luego utilizara el analizador sintactico.

Listing 5: Parte lexica para la calculadora

```
%{
#include "booleanosSintactico.tab.h"
extern int yylval;
%}

%option case-insensitive

%%
true|1          { yylval = 1; return VALOR; }
false|0         { yylval = 0; return VALOR; }
and|"&"         { return AND; }
or|"|"          { return OR; }
not|"!"         { return NOT; }
"("             { return '('; }
")"             { return ')'; }
\n              { return '\n'; }
[ \t]           { /* ignorar espacios en blanco */ }
.               { printf("Caracter no reconocido: %s\n", yytext);
    }
%%

int yywrap() { return 1; }
```

Luego podemos desarrollar la parte sintactica, esta es basicamente la gramatica de la calculadora, aca vamos a definir las reglas que deben seguir las sentencias para ser validas y poder ser procesadas.

Listing 6: Parte sintactica para la calculadora

```
%{
#include <stdio.h>
void yyerror(const char *s);
int yylex(void);
%}

%token VALOR AND OR NOT

%left OR
%left AND
%right NOT

/* Reglas de la gramatica */
%%
calculadora:
    /* vacio */
    | calculadora linea
    ;

linea:
    '\n'
    | expr '\n' { printf("Resultado: %s\n", $1 ? "True" : "False"); }
    ;

expr:
    VALOR            { $$ = $1; }
    | expr OR expr   { $$ = $1 || $3; }
    | expr AND expr  { $$ = $1 && $3; }
    | NOT expr       { $$ = !$2; }
    | '(' expr ')'   { $$ = $2; }
    ;
%%

void yyerror(const char *s) {
    fprintf(stderr, "Error sintactico: %s\n", s);
}

int main(void) {
    printf("Ingrese una expresion booleana (ej. true AND false):\n");
    return yyparse();
}
```

Ejecucuon Calculadora Booleana

```
Ingrese una expresion booleana (true AND false):

false OR NOT false
Resultado: True

true AND (true OR false)
Resultado: True

(true OR false) AND (false AND true)
Resultado: False
```

Estas son pruebas que hice sobre la calculadora para verificar que el funcionamiento fuera correcto y como vemos, sigue bien las reglas y da respuestas correctas.

Pero para entender un poco mejor el desempeño de esta calculadora programada en Bison y por ende de este analizador sintactico, con un archivo llamado *prueba_yacc.py* pude conseguir una grafica sobre el comportamiento en el tiempo de esta calculadora.

El codigo de *prueba_yacc.py* es bastante sencillo. Tiene 2 funciones principales. La primera sirve para crear las cadenas de tamaño n para luego pasarlas como puebas al analizador sintactico.

La segunda funcion es la que se encarga ir haciendo las pruebas a la calculadora con entradas cada vez mas grandes, lo hace mediante una lista que tiene varios valores, que representan el tamaño n de la cadena. A su vez va guardando los tiempos de ejecucion en una lista aparte.

Con esas dos listas obtenemos la siguiente grafica, con la que podemos ver debido a que es Bison tiene un tiempo lineal de ejecucion:

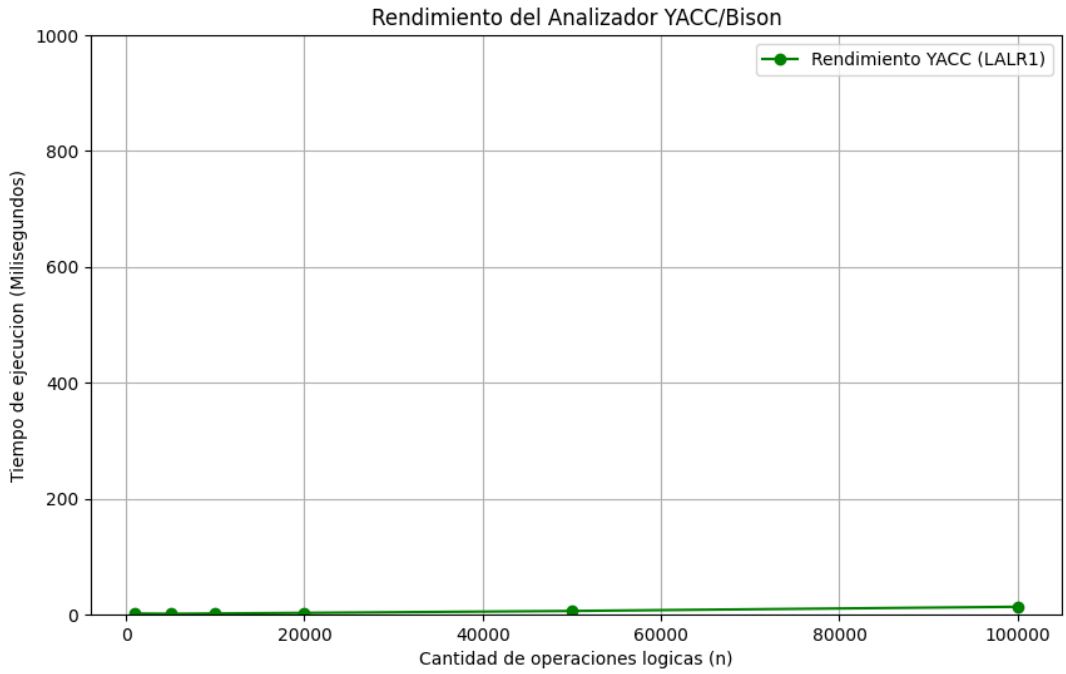

Figure 3: Grafica con tiempo de ejecucion calculadora

Codigo prueba_yacc.py:

Listing 7: Codigo para grafica del rendimiento de la calculadora

```python
import subprocess
import time
import matplotlib.pyplot as plt
import os

def generar_cadena_booleana(n):
    partes = ["true"]
    for i in range(n):
        if i % 2 == 0:
            partes.append("OR false")
        else:
            partes.append("AND true")
    return " ".join(partes) + "\n"

def ejecutar_pruebas_yacc():
    tamanos = [1000, 5000, 10000, 20000, 50000, 100000]
    tiempos = []

    directorio_actual = os.path.dirname(os.path.abspath(__file__))
    ruta_ejecutable = os.path.join(directorio_actual, 'calculadoraBooleana')


    for tam in tamanos:
        cadena = generar_cadena_booleana(tam)

        inicio = time.perf_counter()
        subprocess.run([ruta_ejecutable], input=cadena, text=True, capture_output=True)
        fin = time.perf_counter()
        tiempo_ms = (fin - inicio) * 1000
        tiempos.append(tiempo_ms)

if __name__ == '__main__':
    ejecutar_pruebas_yacc()
```